

Лабораторна робота №5

Тема: Опитування

Студента: Радченка Владислава

Група: 404TN

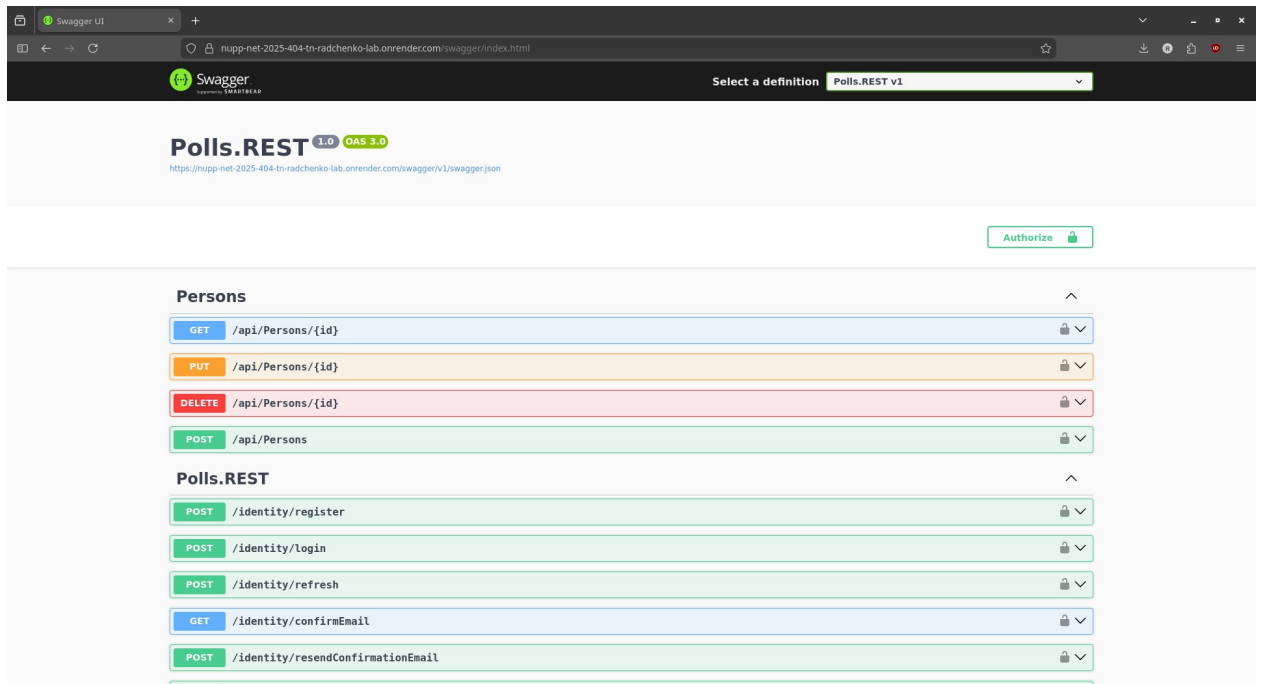


Рисунок 1 – Розгорнутий застосунок сервісом render.com

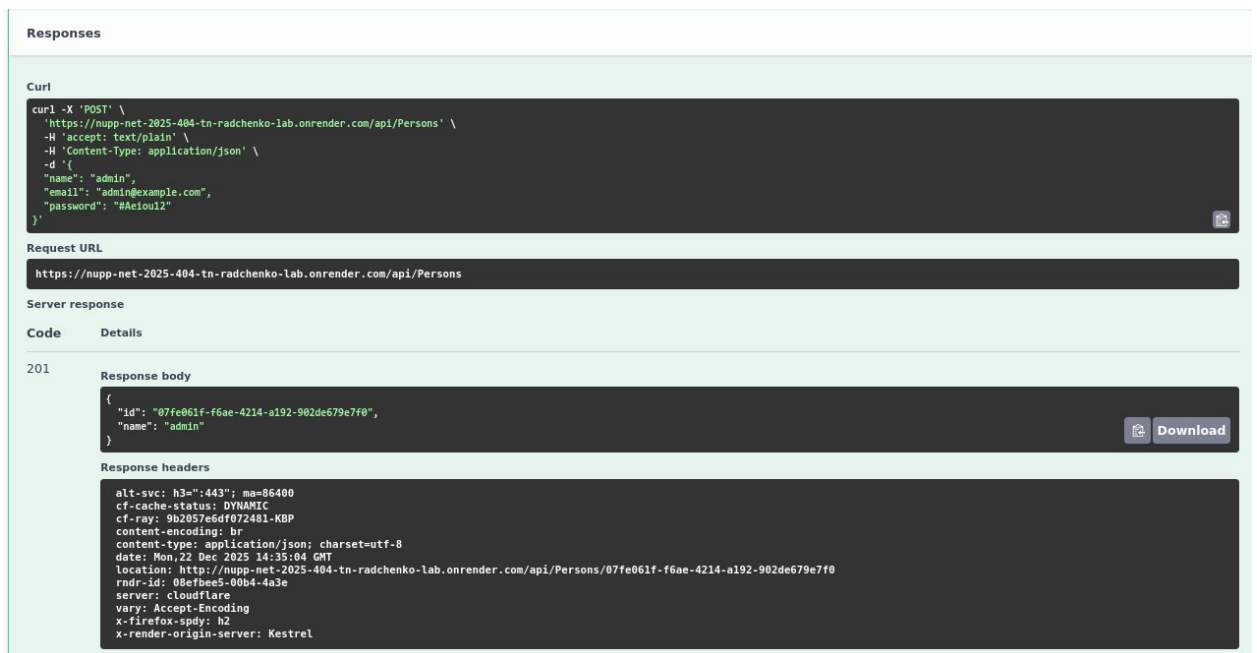


Рисунок 2 – Реєстрація акаунта “Admin”. Акаунти з іменами Admin та Moderator автоматично отримують відповідні ролі.

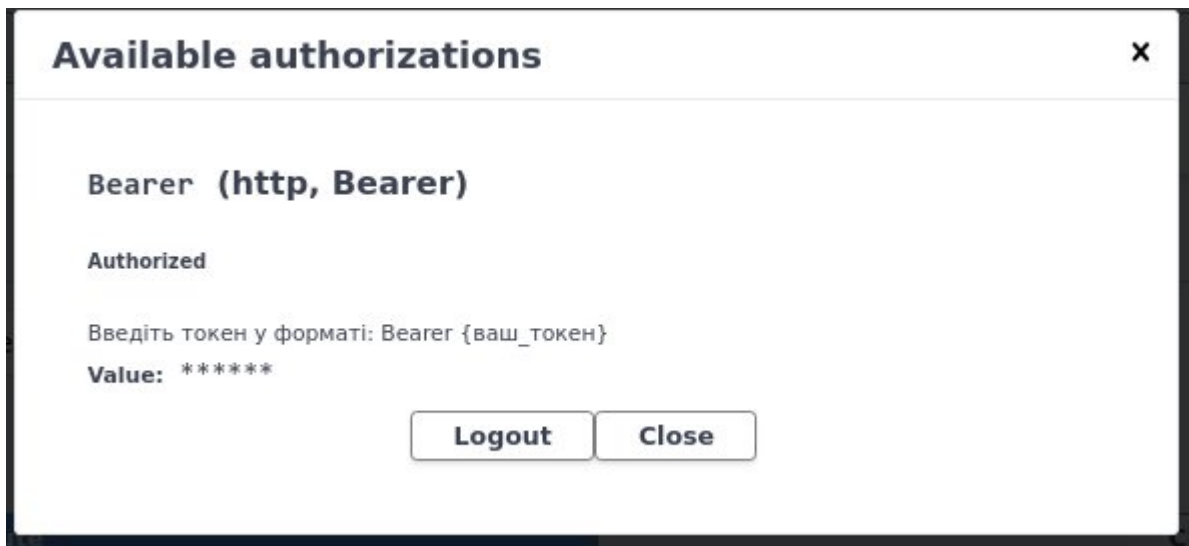


Рисунок 4 – Застосування токена.

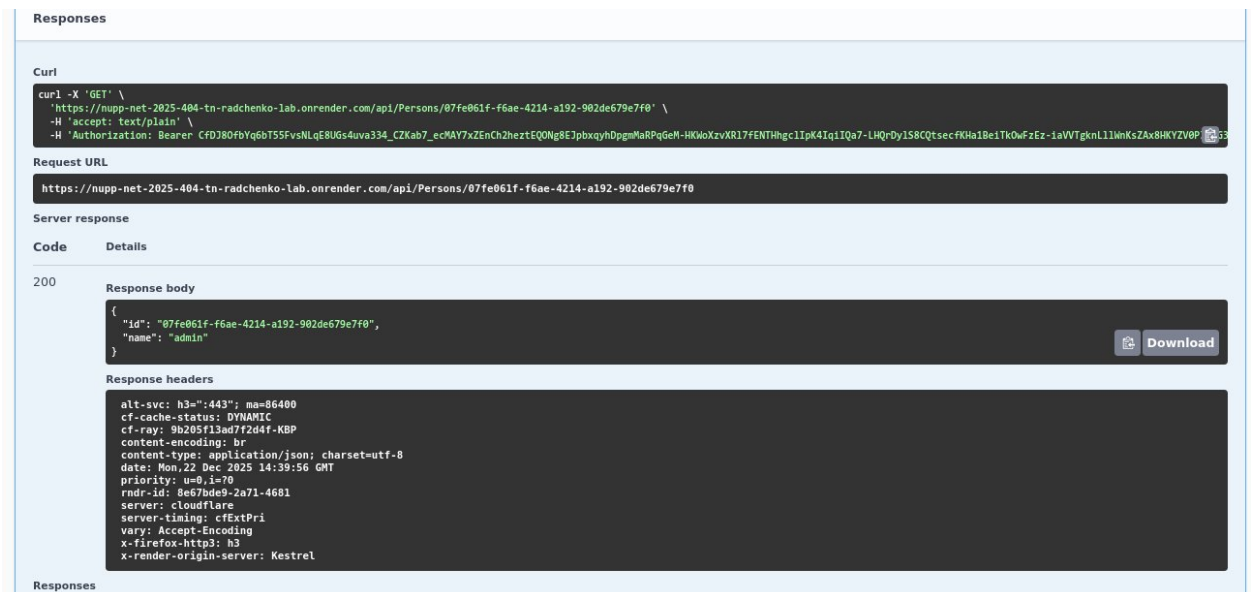


Рисунок 5 – Отримання позитивної відповіді від сервера так як тепер ми авторизовані.

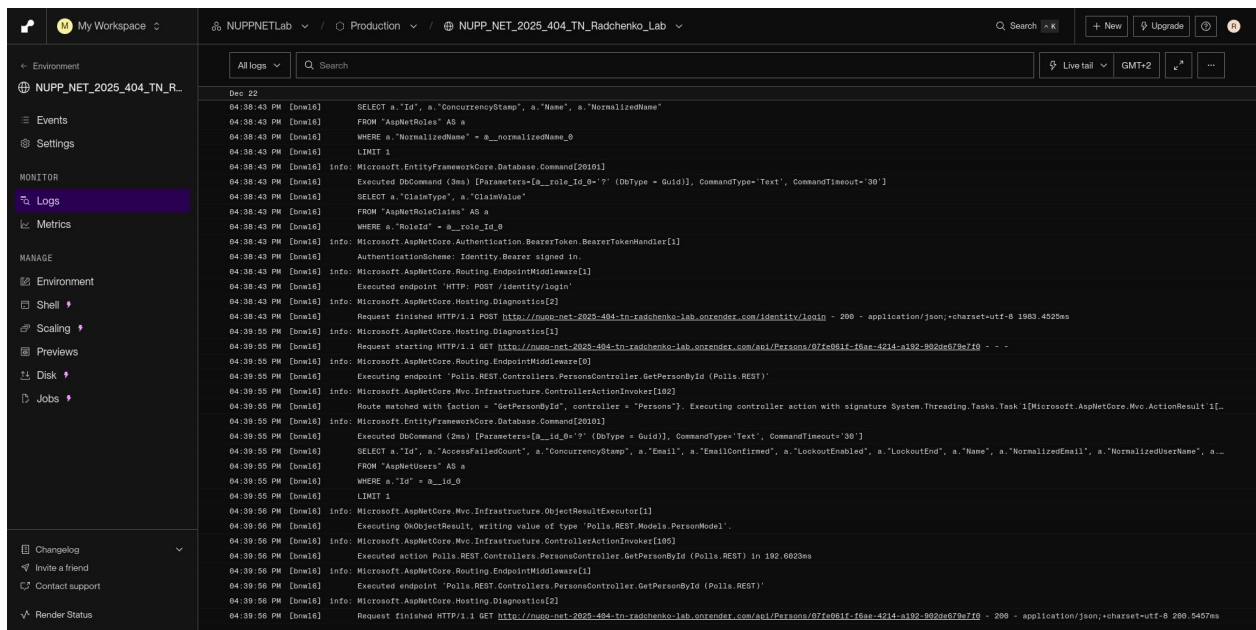


Рисунок 6 – Логи проекту в панелі керування render.com

Для виконання даної лабораторної роботи було реалізовано розподіл прав доступу за трьома ролями: Admin, Moderator та Voter. Для взаємодії з шляхами Person необхідна роль Admin, шляхи відповідальні за опитування потребують Admin або Moderator, а для голосування єдина вимога – бути авторизованим.

Даний проект розміщений за адресою: <https://nupp-net-2025-404-tn-radchenko-lab.onrender.com/swagger/index.html>

Контрольні запитання

1. Для чого в проєкт {Назва тематики}.Infrastructure додається пакет `Microsoft.AspNetCore.Identity.EntityFrameworkCore`, і які можливості він надає?
 - Даний пакет додається для інтеграції системи ідентифікації Microsoft Identity з базою даних через Entity Framework.
2. Яким чином сутність користувача (User) успадковується від `IdentityUser`, та які властивості ви додатково додали до цієї сутності?
 - Успадкування від `IdentityUser` додає нові поля необхідні для працездатності авторизації, а саме: `Id`, `UserName`, `Email`, `PasswordHash`, `SecurityStamp`, тощо.
3. Чому `DbContext` потрібно наслідувати від `IdentityDbContext<TUser>`, і які зміни це вносить у конфігурацію бази даних?
 - Це необхідно робити для додання нових 7 таблиць які створюються та керуються автоматично: `AspNetUsers`, `AspNetRoles`, `AspNetUserRoles`, `AspNetRoleClaims`, `AspNetUserClaims`, `AspNetUserLogins`, `AspNetUserTokens`.
4. Як налаштовуються служби Identity у проєкті {Назва тематики}.REST, зокрема менеджери користувачів (`UserManager`) та ролей (`RoleManager`)?
 - Для налаштування служби Identity використовується метод `AddIdentityApiEndpoints<T>()`
5. Які кроки потрібні для створення та підписування JWT або Bearer-токенів в ASP.NET Core?
 1. Створення об'єкта Claims
 2. Вибір алгоритму шифрування
 3. Використання секретного ключа для створення цифрового підпису
 4. Встановлення терміну дії
6. Яким чином налаштовується Middleware для автентифікації та авторизації на основі JWT у вашому REST API?
 - Middleware для автентифікації та авторизації налаштовується додаванням `app.UseAuthentication();` та `app.UseAuthorization();`. Для визначення прав доступу вказуємо `[Authorize]` над методами контролерів.
7. Що містить HTTP-заголовок `Authorization`, і як його обробляє сервер при отриманні запиту?
 - HTTP-заголовок `Authorization` містить токен, що використовується сервером для ідентифікації користувача.
8. Які кінцеві точки REST API залишилися загальнодоступними, а які захищені — та чому саме такий розподіл? Наведіть приклади.
 - Як вже було згадано, для виконання даної лабораторної роботи було реалізовано розподіл прав доступу за трьома ролями: `Admin`, `Moderator` та `Voter`. Для взаємодії з шляхами `Person` необхідна роль `Admin`, шляхи відповідальні за опитування потребують `Admin` або `Moderator`, а для голосування єдина вимога – бути авторизованим.

- Такий розподіл було обрано для надання можливості гостям проекту переглядати опитування та їх результати доки авторизовані користувачі вже отримують здатність впливати на об'єкти дивлячись на їх рівень доступу. Лише адміністратори та модератори повинні мати можливість впливати на користувачів та їх дії.
9. Як ви реалізували поділ повноважень між ролями (мінімум три ролі) та якими атрибутами (наприклад, `[Authorize(Roles = "...")]`) це забезпечується?
 - Відповідь дана в попередньому пункті. Для реалізації було використано атрибутути `[Authorize]`, `[Authorize(Roles="Admin")]` та `[Authorize(Roles="Admin,Moderator")]`
 10. Як обробляються ситуації, коли користувачі неавторизовані або мають недостатні права? Які HTTP-коди повертаються в цих випадках?
 - Неавтентифікованим користувачам та тим, хто має недостатній рівень доступу повертаються помилки 401 Unauthorized та 403 Forbidden відповідно.
 11. У чому відмінність між автентифікацією та авторизацією, і як ці процеси взаємодіють у вашому застосунку?
 - Автентифікація визначає користувача, тоді як авторизація, його рівень доступу.
 12. Як ви тестували кінцеві точки реєстрації, автентифікації та авторизованих запитів? Які приклади потрібно включити в PDF-звіт?
 - Тестування проводилося шляхом створення нових акаунтів та перевіркою працездатності REST з їх токенами.
 13. Які кроки виконані для розгортання застосунку у вибраній хмарній/хостинговій платформі, і які сервіси були задіяні?
 - Для розгортання було створено Dockerfile та завантажено останню версію коду на GitHub. Після цього було створено новий проект в сервісі render.com, де було вказано посилання на репозиторій та створено базу даних для нього. Сервіс завантажив репозиторій та розгорнув код, в результаті чого було отримано розгорнутий проект.
 14. Як ви налаштували перенос налаштувань (connection string, JWT-секрети тощо) у середовище розгортання, та як забезпечили безпеку цих даних?
 - В коді реалізована автоматична міграція, дані з локального середовища не були вивантажені хоча при необхідності це можна було б зробити інструментом `pg_dump`. Перенос connection string відбувся створенням нової бази даних сервісом render.com, що повернув рядок з'єднання для internal з'єднань. Цей рядок я потім вказав як значення змінної середовища `ConnectionStrings__DefaultConnection`, яка потім завантажується конструктором в першу чергу перед завантаженням з файлу.
 15. Які перевірки виконувалися, щоб упевнитися, що розгорнута версія застосунку працює коректно, та які матеріали (посилання, скріншоти) потрібно додати до звіту?

- Проект було розгорнуто через Docker, що зрівнює середовища виконання. Єдиним хвилюванням лишалось з'єднання з базою даних та міграція, але позитивний результат успішного з'єднання можливо побачити в консолі з логами, що і сталося.